

Proyecto Fase 1

■ Fecha	@10 de septiembre de 2025
■ Status	En Proceso

Integrantes Grupo 5

Nombre	Registro Académico
Josseline Griselda Montecinos Hernández	202201534
Pablo Andrés Rodríguez Lima	202201947
Carlos Manuel Lima y Lima	202201524

ARCHIVOS DESCARGADOS

title.akas.tsv.gz - Títulos Alternativos/Localizados

Campo	Qué Guarda
titleId	Identificador único alfanumérico del título (tconst) - Clave foránea a títulos principales
ordering	Número entero para identificar únicamente cada fila por titleId - Manejo de múltiples versiones
title	Título localizado/alternativo - Cómo se conoce el título en diferentes regiones
region	Código de región/país donde se usa esta versión del título (ISO)
language	Idioma del título alternativo (códigos ISO de idioma)
types	Array de atributos del título alternativo: "alternative", "dvd", "festival", "tv", "video", "working", "original", "imdbDisplay"
attributes	Términos adicionales descriptivos no enumerados - Información extra sobre el título alternativo
isOriginalTitle	Booleano (0/1) - Indica si es el título original de la obra

title.basics.tsv.gz - Información Básica de Títulos

Campo	Qué Guarda
tconst	Identificador único alfanumérico del título - Clave primaria universal de títulos
titleType	Tipo/formato del título: movie, short, tvseries, tvepisode, video, etc. - Categorización del contenido
primaryTitle	Título más popular/usado por realizadores en materiales promocionales - Título de exhibición principal
originalTitle	Título original en el idioma original de producción
isAdult	Contenido para adultos (0: no adulto; 1: adulto) - Clasificación de audiencia
startYear	Año de lanzamiento (YYYY). Para series TV es el año de inicio
endYear	Año de finalización para series TV (YYYY). '\N' para otros tipos de título
runtimeMinutes	Duración principal del título en minutos
genres	Array de hasta tres géneros asociados con el título - Clasificación temática

title.crew.tsv.gz - Equipo de Producción Principal

Campo	Qué Guarda
tconst	Identificador único alfanumérico del título - Referencia al título
directors	Array de identificadores de personas (nconsts) que dirigieron el título
writers	Array de identificadores de personas (nconsts) que escribieron/guionizaron el título

title.episode.tsv.gz - Información de Episodios

Campo	Qué Guarda
tconst	Identificador alfanumérico del episodio específico - Clave primaria del episodio
parentTconst	Identificador alfanumérico de la serie TV padre - Relación jerárquica serie→episodio
seasonNumber	Número entero de temporada a la que pertenece el episodio
episodeNumber	Número entero del episodio dentro de la serie TV

title.principals.tsv.gz - Reparto y Equipo Principal

Campo	Qué Guarda
tconst	Identificador único alfanumérico del título - Referencia al título
ordering	Número entero para identificar únicamente filas por titleId - Orden de importancia en créditos
nconst	Identificador único alfanumérico de la persona - Referencia a base de datos de personas
category	Categoría del trabajo realizado por la persona (actor, director, writer, producer, etc.)
job	Título específico del trabajo si aplica, sino '\N' - Trabajo detallado (ej: "Executive Producer")
characters	Nombre del personaje interpretado si aplica, sino '\N' - Solo para actores/actrices

title.ratings.tsv.gz - Calificaciones y Votaciones

Campo	Qué Guarda
tconst	Identificador único alfanumérico del título - Referencia al título calificado
averageRating	Promedio ponderado de todas las calificaciones individuales de usuarios (escala 1-10)
numVotes	Número total de votos que ha recibido el título - Indicador de popularidad

name.basics.tsv.gz - Información de Personas

Campo	Qué Guarda
nconst	Identificador único alfanumérico de la persona - Clave primaria de personas
primaryName	Nombre por el cual la persona es más frecuentemente acreditada - Nombre profesional
birthYear	Año de nacimiento en formato YYYY
deathYear	Año de muerte en formato YYYY si aplica, sino '\N' - Para personas fallecidas
primaryProfession	Array de strings con las top-3 profesiones de la persona - Actividades principales
knownForTitles	Array de identificadores de títulos (tconsts) por los cuales la persona es conocida - Trabajos más relevantes

TABLAS

Categoría

Campo	Que Guarda	
id_categoria	Identificador único	
nombre	nombre de la categoría	Tipo/formato del título: movie, short, tvseries, tvepisode, video, etc. - Categorización del contenido

Género

Campo	Que Guarda	
id_genero	Identificador único	
nombre	nombre del genero	Comedia, Drama, Acción, etc

Título

Campo	Qué Guarda
tconst (PK)	Identificador único alfanumérico del título - Clave primaria universal de títulos
id_categoria	Llave foránea a categoría
titulo_popular	Título más popular/usado por realizadores en materiales promocionales - Título de exhibición principal
titulo_original	Título original en el idioma original de producción
es_contenido_adulto	Contenido para adultos (0: no adulto; 1: adulto) - Clasificación de audiencia
anio_lanzamiento	Año de lanzamiento (YYYY). Para series TV es el año de inicio
anio_finalizacion	Año de finalización para series TV (YYYY). '\N' para otros tipos de título
duracion_minutos	Duración principal del título en minutos
generos	Lista de generos

Título_Alternativo

Campo	Qué Guarda
Id_tituloAlternativo (PK)	Identificador Unico
tconst (FK)	Identificador único alfanumérico del título - Referencia al título
orden	número para identificar de forma única las filas para un titulo determinado
nombre_titulo	nombre del titulo segun la region disponible
region	la región para esta versión del título
idioma	el idioma del título
tipo	Conjunto enumerado de atributos para este título alternativo.
atributo	Términos adicionales para describir este título alternativo, no enumerados
es_Original	0: título no original; 1: título original

Equipo de Producción

Campo	Qué Guarda
id_produccion	Identificador único
tconst (FK)	Identificador único alfanumérico del título - Referencia al título
directores	Array de identificadores de personas (nconsts) que dirigieron el título
escritores	Array de identificadores de personas (nconsts) que escribieron/guionizaron el título

Episodio

Campo	Qué Guarda
tconst	Identificador alfanumérico del episodio específico - Clave primaria del episodio
id_titulo FK	Identificador alfanumérico de la serie TV padre - Relación jerárquica serie→episodio
temporada	Número entero de temporada a la que pertenece el episodio
episodio	Número entero del episodio dentro de la serie TV

Reparto

Campo	Qué Guarda
id_reparto	Identificador único
tconst FK	Identificador único alfanumérico del título - Referencia al título
relevancia	Número entero para identificar únicamente filas por titleId - Orden de importancia en créditos
nconst FK	Identificador único alfanumérico de la persona - Referencia a base de datos de personas
categoría	Categoría del trabajo realizado por la persona (actor, director, writer, producer, etc.)
Rol	Título específico del trabajo si aplica, sino '\N' - Trabajo detallado (ej: "Executive Producer")
Personaje	Nombre del personaje interpretado si aplica, sino '\N' - Solo para actores/actrices

Puntuacion

Campo	Qué Guarda
id_puntuacion	
tconst FK	Identificador único alfanumérico del título - Referencia al título calificado
promedio	Promedio ponderado de todas las calificaciones individuales de usuarios (escala 1-10)
votos	Número total de votos que ha recibido el título - Indicador de popularidad

Personas

Campo	Qué Guarda
nconst PK	Identificador único alfanumérico de la persona - Clave primaria de personas

Campo	Qué Guarda
nombre_artistico	Nombre por el cual la persona es más frecuentemente acreditada - Nombre profesional
anio_nacimiento	Año de nacimiento en formato YYYY
anio_fallecimiento	Año de muerte en formato YYYY si aplica, sino '\N' - Para personas fallecidas
Profesiones Principales	Array de strings con las top-3 profesiones de la persona - Actividades principales
Conocido Por Titulos	Array de identificadores de títulos (tconsts) por los cuales la persona es conocida - Trabajos más relevantes

PRIMERA FORMA NORMAL

Objetivo: Eliminar grupos repetitivos y garantizar valores atómicos

Categoría

Campo	Que Guarda	
id_categoria	Identificador único	
nombre	nombre de la categoría	Tipo/formato del título: movie, short, tvseries, tvepisode, video, etc. - Categorización del contenido

Género

Campo	Que Guarda	
id_genero	Identificador único	
nombre	nombre del genero	Comedia, Drama, Acción, etc

Titulo

- **ELIMINADO:** Campo `generos` (contenía múltiples valores separados por comas)
- **CREADA:** Tabla `Genero_Titulo` para manejar la relación muchos-a-muchos entre títulos y géneros
- **Razón:** El campo `generos` violaba la 1FN al contener múltiples valores en una sola celda

Campo	Qué Guarda
tconst (PK)	Identificador único alfanumérico del título - Clave primaria universal de títulos
id_categoria	Llave foránea a categoría
titulo_popular	Título más popular/usado por realizadores en materiales promocionales - Título de exhibición principal
titulo_original	Título original en el idioma original de producción
es_contenido_adulto	Contenido para adultos (0: no adulto; 1: adulto) - Clasificación de audiencia
anio_lanzamiento	Año de lanzamiento (YYYY). Para series TV es el año de inicio
anio_finalizacion	Año de finalización para series TV (YYYY). '\N' para otros tipos de título
duracion_minutos	Duración principal del título en minutos

Genero_Titulo

Campo	Qué Guarda
id_genero_titulo (PK)	Identificador único
tconst (FK)	Identificador único alfanumérico del título - Clave primaria universal de títulos
id_genero (FK)	Genero del titulo

Titulo_Alternativo

Campo	Qué Guarda
Id_tituloAlternativo (PK)	Identificador único
tconst (FK)	Identificador único alfanumérico del título - Referencia al título
orden	Número para identificar de forma única las filas para un título determinado
nombre_titulo	Nombre del título según la región disponible
region	La región para esta versión del título
idioma	El idioma del título
tipo	Conjunto enumerado de atributos para este título alternativo
atributo	Términos adicionales para describir este título alternativo, no enumerados
es_Original	0: título no original; 1: título original

Equipo de Producción

- **ELIMINADO:** Campo `directores` (contenía array de nconsts)
- **ELIMINADO:** Campo `escritores` (contenía array de nconsts)
- **CREADA:** Tabla `Produccion_Escritor` para descomponer el array de escritores
- **CREADA:** Tabla `Produccion_Director` para descomponer el array de directores
- **MODIFICADA:** Se mantuvo tabla `Produccion` como tabla base

Campo	Qué Guarda
id_produccion (PK)	Identificador único
tconst (FK)	Identificador único alfanumérico del título - Referencia al título

Produccion_Escritor

Campo	Qué Guarda	
id_produccion_escritor (PK)	Identificador único	
id_persona (FK)	Llave Foranea Persona	
id_produccion (FK)	Llave Foranea Produccion	

Produccion_Director

Campo	Qué Guarda	
id_produccion_director (PK)	Identificador único	
id_persona (FK)	Llave Foranea Persona	
id_produccion (FK)	Llave Foranea Produccion	

Episodio

Campo	Qué Guarda
tconst (PK)	Identificador alfanumérico del episodio específico - Clave primaria del episodio
id_titulo (FK)	Identificador alfanumérico de la serie TV padre - Relación jerárquica serie→episodio
temporada	Número entero de temporada a la que pertenece el episodio
episodio	Número entero del episodio dentro de la serie TV

Reparto

MODIFICADO: Campo `nconst` cambió de FK a campo directo (preparación para normalización posterior)

Campo	Qué Guarda
id_reparto (PK)	Identificador único
tconst (FK)	Identificador único alfanumérico del título - Referencia al título
relevancia	Número entero para identificar únicamente filas por titleId - Orden de importancia en créditos
nconst (FK)	Identificador único alfanumérico de la persona - Referencia a base de datos de personas
categoría	Categoría del trabajo realizado por la persona (actor, director, writer, producer, etc.)
Rol	Título específico del trabajo si aplica, sino '\N' - Trabajo detallado (ej: "Executive Producer")
Personaje	Nombre del personaje interpretado si aplica, sino '\N' - Solo para actores/actrices

Puntuacion

Campo	Qué Guarda
id_puntuacion (PK)	Identificador único
tconst (FK)	Identificador único alfanumérico del título - Referencia al título calificado
promedio	Promedio ponderado de todas las calificaciones individuales de usuarios (escala 1-10)
votos	Número total de votos que ha recibido el título - Indicador de popularidad

Persona

- **ELIMINADO:** Campo `Profesiones Principales` (contenía array de profesiones)
- **ELIMINADO:** Campo `Conocido Por Titulos` (contenía array de tconsts)
- **CREADA:** Tabla `Profesion_Persona_Top` para las profesiones principales
- **CREADA:** Tabla `Titulo_Persona_Top` para los títulos por los que es conocida

Campo	Qué Guarda
nconst (PK)	Identificador único alfanumérico de la persona - Clave primaria de personas
nombre_artistico	Nombre por el cual la persona es más frecuentemente acreditada - Nombre profesional

Campo	Qué Guarda
anio_nacimiento	Año de nacimiento en formato YYYY
anio_fallecimiento	Año de muerte en formato YYYY si aplica, sino '\N' - Para personas fallecidas

Profesion_Persona_Top

Campo		
id_profesion_persona	Identificador único	
nconst (FK)	persona	
tconst (FK)	Profesión principal de la persona (sin normalizar)	

Pelicula_Persona_Top

Campo		
id_profesion_persona	Identificador único	
nconst (FK)	persona	
tconst (FK)	Título por el cual es conocida la persona	

SEGUNDA FORMA NORMAL

Categoría

Campo	Que Guarda	
id_categoria	Identificador único	
nombre	nombre de la categoría	Tipo/formato del título: movie, short, tvseries, tvepisode, video, etc. - Categorización del contenido

Género

Campo	Que Guarda	
id_genero	Identificador único	
nombre	nombre del genero	Comedia, Drama, Acción, etc

Profesion

- **Origen:** Nace de la necesidad de normalizar el campo `categoria` en la tabla Reparto
- **Razón:** El campo `categoria` contenía valores repetitivos (actor, director, writer, producer) que debían normalizarse para evitar redundancia y garantizar consistencia

Campo	Qué Guarda: Actor, Director,
id_profesion (PK)	Identificador único
nombre	nombre de la profesión

Titulo

Campo	Qué Guarda
tconst (PK)	Identificador único alfanumérico del título - Clave primaria universal de títulos
id_categoria (FK)	Llave foránea a categoría
titulo_popular	Título más popular/usado por realizadores en materiales promocionales - Título de exhibición principal
titulo_original	Título original en el idioma original de producción
es_contenido_adulto	Contenido para adultos (0: no adulto; 1: adulto) - Clasificación de audiencia
anio_lanzamiento	Año de lanzamiento (YYYY). Para series TV es el año de inicio
anio_finalizacion	Año de finalización para series TV (YYYY). '\N' para otros tipos de título
duracion_minutos	Duración principal del título en minutos

Genero_Titulo

Campo	Qué Guarda
id_genero_titulo (PK)	Identificador único
tconst (FK)	Identificador único alfanumérico del título - Clave primaria universal de títulos
id_genero (FK)	Genero del titulo

Titulo_Alternativo

- Cambiar campos repetitivos por **FK** (id_region, id_idioma, id_tipo)

Campo	Qué Guarda
Id_tituloAlternativo (PK)	Identificador único
tconst (FK)	Referencia al título
orden	Número de orden del título alternativo
nombre_titulo	Nombre del título según la región
id_region (FK)	Referencia a la región
id_idioma (FK)	Referencia al idioma
id_tipo (FK)	Referencia al tipo de título
atributo	Términos adicionales descriptivos
es_Original	Indica si es título original

Region

- Separar **region** → tabla **Region** independiente

Campo	Qué Guarda
id_region (PK)	Identificador único de región
codigo	Código de región (DE, US, HU, GR)

Idioma

- Separar **idioma** → tabla **Idioma** independiente

Campo	Qué Guarda
id_idioma (PK)	Identificador único de idioma

Campo	Qué Guarda
codigo	Código de idioma (EN, ES, etc.)

Tipo_Titulo

- Separar **tipo** → tabla **Tipo_Titulo** independiente

Campo	Qué Guarda
id_tipo (PK)	Identificador único de tipo
nombre	Nombre del tipo (original, imdbDisplay)

Equipo de Producción

Campo	Qué Guarda
id_produccion (PK)	Identificador único
tconst (FK)	Identificador único alfanumérico del título - Referencia al título

Produccion_Escritor

Campo	Qué Guarda	
id_produccion_escritor (PK)	Identificador único	
id_persona (FK)	Llave Foranea Persona	
id_produccion (FK)	Llave Foranea Produccion	

Produccion_Director

Campo	Qué Guarda	
id_produccion_director (PK)	Identificador único	
id_persona (FK)	Llave Foranea Persona	
id_produccion (FK)	Llave Foranea Produccion	

Episodio

Campo	Qué Guarda
tconst (PK)	Identificador alfanumérico del episodio específico - Clave primaria del episodio
id_titulo (FK)	Identificador alfanumérico de la serie TV padre - Relación jerárquica serie→episodio
temporada	Número entero de temporada a la que pertenece el episodio
episodio	Número entero del episodio dentro de la serie TV

Reparto

- **ELIMINADO:** Campo `categoria` (contenía texto directo como "actor", "director")
- **AGREGADO:** Campo `id_profesion (FK)` que referencia a la nueva tabla Profesion

Campo	Qué Guarda
id_reparto (PK)	Identificador único
tconst (FK)	Identificador único alfanumérico del título - Referencia al título
relevancia	Número entero para identificar únicamente filas por titleId - Orden de importancia en créditos

Campo	Qué Guarda
nconst (FK)	Identificador único alfanumérico de la persona - Referencia a base de datos de personas
id_profesion (FK)	Referencia a profesión
Rol	Título específico del trabajo si aplica, sino '\N' - Trabajo detallado (ej: "Executive Producer")
Personaje	Nombre del personaje interpretado si aplica, sino '\N' - Solo para actores/actrices

Puntuacion

Campo	Qué Guarda
id_puntuacion (PK)	Identificador único
tconst (FK)	Identificador único alfanumérico del título - Referencia al título calificado
promedio	Promedio ponderado de todas las calificaciones individuales de usuarios (escala 1-10)
votos	Número total de votos que ha recibido el título - Indicador de popularidad

Persona

Campo	Qué Guarda
nconst (PK)	Identificador único alfanumérico de la persona - Clave primaria de personas
nombre_artistico	Nombre por el cual la persona es más frecuentemente acreditada - Nombre profesional
anio_nacimiento	Año de nacimiento en formato YYYY
anio_fallecimiento	Año de muerte en formato YYYY si aplica, sino '\N' - Para personas fallecidas

Profesion_Persona_Top

- **ELIMINADO:** Campo `profesion` (contenía texto directo de profesión)
- **AGREGADO:** Campo `id_profesion (FK)` que referencia a la tabla Profesion
- **RAZÓN:** Normalizar las profesiones para evitar duplicación de datos

Campo		
id_profesion_persona	Identificador único	
nconst (FK)	persona	
id_profesion (FK)		

Pelicula_Persona_Top

Campo		
id_profesion_persona	Identificador único	
nconst (FK)	persona	
tconst (FK)	id del titulo	

TERCERA FORMA NORMAL

ANÁLISIS: No se requirieron cambios estructurales

- **RAZÓN:** La estructura de 2FN ya cumplía con los requisitos de 3FN
- **VERIFICACIÓN:** No existen dependencias transitivas en el diseño
 - Todos los atributos no clave dependen directamente de sus claves primarias
 - No hay atributos que dependan de otros atributos no clave

Categoria

Campo	Descripción del Campo	Nombre Original en Inglés	Tipo de Datos
id_categoria	Identificador único de categoría	<i>Campo artificial - no existe en archivos</i>	SERIAL PRIMARY KEY
nombre	Tipo/formato del título	<code>titleType</code> (de title.basics.tsv.gz)	VARCHAR(50) NOT NULL

Genero

Campo	Descripción del Campo	Nombre Original en Inglés	Tipo de Datos
id_genero	Identificador único de género	<i>Campo artificial - no existe en archivos</i>	SERIAL PRIMARY KEY
nombre	Nombre del género cinematográfico	<code>genres</code> (extraído de title.basics.tsv.gz)	VARCHAR(50) NOT NULL

Titulo

Campo	Descripción del Campo	Nombre Original en Inglés	Tipo de Datos
tconst	Identificador único del título (PK)	<code>tconst</code> (de title.basics.tsv.gz)	VARCHAR(20) PRIMARY KEY
id_categoria	Llave foránea a categoría	<code>titleType</code> (de title.basics.tsv.gz)	INTEGER REFERENCES Categoria(id_categoria)
titulo_popular	Título principal/más conocido	<code>primaryTitle</code> (de title.basics.tsv.gz)	TEXT NOT NULL
titulo_original	Título en idioma original	<code>originalTitle</code> (de title.basics.tsv.gz)	TEXT
es_contenido_adulto	Indicador de contenido adulto	<code>isAdult</code> (de title.basics.tsv.gz)	BOOLEAN
anio_lanzamiento	Año de lanzamiento o inicio	<code>startYear</code> (de title.basics.tsv.gz)	INTEGER
anio_finalizacion	Año de finalización (series)	<code>endYear</code> (de title.basics.tsv.gz)	INTEGER
duracion_minutos	Duración en minutos	<code>runtimeMinutes</code> (de title.basics.tsv.gz)	INTEGER

Genero_Titulo

Campo	Descripción del Campo	Nombre Original en Inglés	Tipo de Datos
id_genero_titulo	Identificador único (PK)	<i>Campo artificial - no existe en archivos</i>	SERIAL PRIMARY KEY

Campo	Descripción del Campo	Nombre Original en Inglés	Tipo de Datos
tconst	Referencia al título (FK)	tconst (de title.basics.tsv.gz)	VARCHAR(20) REFERENCES Título(tconst)
id_genero	Referencia al género (FK)	genres (procesado de title.basics.tsv.gz)	INTEGER REFERENCES Genero(id_genero)

Titulo_Alternativo

- Cambiar campo atributo por FK (id_atributo)

Campo	Descripción	Tipo de Dato
Id_tituloAlternativo	Identificador único (PK)	INT
tconst	Referencia al título (FK)	VARCHAR(10)
orden	Número de orden	INT
nombre_titulo	Nombre del título	VARCHAR(255)
id_region	Referencia a región (FK)	INT
id_idioma	Referencia a idioma (FK)	INT
es_Original	Indica si es original	BOOLEAN

Titulo_Alternativo_Atributo

Campo	Descripción del Campo	Tipo de Datos
id_Titulo_Alternativo_Atributo	Identificador único (PK)	SERIAL PRIMARY KEY
id_titulo_alternativo	Referencia a Título_Alternativo (FK)	INT REFERENCES Título_Alternativo(Id_tituloAlternativo)
id_atributo	Referencia a Atributo (FK)	INT REFERENCES Atributo(id_atributo)

Título_Alternativo_Tipo

Campo	Descripción del Campo	Tipo de Datos
id_Titulo_Alternativo_Atributo	Identificador único (PK)	SERIAL PRIMARY KEY
id_titulo_alternativo	Referencia a Título_Alternativo (FK)	INT REFERENCES Título_Alternativo(Id_tituloAlternativo)
id_tipo	Referencia a Tipo_Título (FK)	INT REFERENCES Tipo_Título(id_tipo)

Region

Campo	Descripción	Tipo de Dato
id_region	Identificador único (PK)	INT
codigo	Código de región	VARCHAR(5)

Idioma

Campo	Descripción	Tipo de Dato
id_idioma	Identificador único (PK)	INT
codigo	Código de idioma	VARCHAR(5)

Tipo_Titulo

Campo	Descripción	Tipo de Dato
id_tipo	Identificador único (PK)	INT
nombre	Nombre del tipo	VARCHAR(50)

Atributo

- Separar **atributo** → tabla **Atributo** independiente

Campo	Descripción	Tipo de Dato
id_atributo	Identificador único (PK)	INT
nombre	Nombre del atributo	VARCHAR(100)

Produccion

Campo	Descripción del Campo	Nombre Original en Inglés	Tipo de Datos
id_produccion	Identificador único (PK)	<i>Campo artificial - no existe en archivos</i>	SERIAL PRIMARY KEY
tconst	Referencia al título (FK)	tconst (de title.crew.tsv.gz)	VARCHAR(20) REFERENCES Titulo(tconst)

Produccion_Escritor

Campo	Descripción del Campo	Nombre Original en Inglés	Tipo de Datos
id_produccion_escritor	Identificador único (PK)	<i>Campo artificial - no existe en archivos</i>	SERIAL PRIMARY KEY
id_persona	Referencia a persona (FK)	writers (procesado de title.crew.tsv.gz)	VARCHAR(20) REFERENCES Persona(nconst)
id_produccion	Referencia a producción (FK)	tconst (de title.crew.tsv.gz)	INTEGER REFERENCES Produccion(id_produccion)

Produccion_Director

Campo	Descripción del Campo	Nombre Original en Inglés	Tipo de Datos
id_produccion_director	Identificador único (PK)	<i>Campo artificial - no existe en archivos</i>	SERIAL PRIMARY KEY
id_persona	Referencia a persona (FK)	directors (procesado de title.crew.tsv.gz)	VARCHAR(20) REFERENCES Persona(nconst)
id_produccion	Referencia a producción (FK)	tconst (de title.crew.tsv.gz)	INTEGER REFERENCES Produccion(id_produccion)

Episodio

Campo	Descripción del Campo	Nombre Original en Inglés	Tipo de Datos
tconst	Identificador del episodio (PK)	tconst (de title.episode.tsv.gz)	VARCHAR(20) PRIMARY KEY
id_titulo	Referencia a serie padre (FK)	parentTconst (de title.episode.tsv.gz)	VARCHAR(20) REFERENCES Titulo(tconst)
temporada	Número de temporada	seasonNumber (de title.episode.tsv.gz)	INTEGER
episodio	Número de episodio	episodeNumber (de title.episode.tsv.gz)	INTEGER

Profesion

Campo	Descripción del Campo	Nombre Original en Inglés	Tipo de Datos
id_profesion	Identificador único (PK)	Campo artificial - no existe en archivos	SERIAL PRIMARY KEY
nombre	Nombre de la profesión	category (de title.principals.tsv.gz) y primaryProfession (de name.basics.tsv.gz)	VARCHAR(100) NOT NULL

Reparto

Campo	Descripción del Campo	Nombre Original en Inglés	Tipo de Datos
id_reparto	Identificador único (PK)	Campo artificial - no existe en archivos	SERIAL PRIMARY KEY
tconst	Referencia al título (FK)	tconst (de title.principals.tsv.gz)	VARCHAR(20) REFERENCES Titulo(tconst)
relevancia	Orden de importancia en créditos	ordering (de title.principals.tsv.gz)	INTEGER
nconst	Referencia a persona (FK)	nconst (de title.principals.tsv.gz)	VARCHAR(20) REFERENCES Persona(nconst)
id_profesion	Referencia a profesión (FK)	category (de title.principals.tsv.gz)	INTEGER REFERENCES Profesion(id_profesion)
rol	Trabajo específico detallado	job (de title.principals.tsv.gz)	TEXT
personaje	Personaje interpretado	characters (de title.principals.tsv.gz)	TEXT

Puntuacion

Campo	Descripción del Campo	Nombre Original en Inglés	Tipo de Datos
id_puntuacion	Identificador único (PK)	Campo artificial - no existe en archivos	SERIAL PRIMARY KEY
tconst	Referencia al título (FK)	tconst (de title.ratings.tsv.gz)	VARCHAR(20) REFERENCES Titulo(tconst)
promedio	Calificación promedio (1.0-10.0)	averageRating (de title.ratings.tsv.gz)	DECIMAL(5,3)
votos	Número total de votos	numVotes (de title.ratings.tsv.gz)	INTEGER

Persona

Campo	Descripción del Campo	Nombre Original en Inglés	Tipo de Datos
nconst	Identificador único de persona (PK)	nconst (de name.basics.tsv.gz)	VARCHAR(20) PRIMARY KEY
nombre_artistico	Nombre profesional/artístico	primaryName (de name.basics.tsv.gz)	TEXT NOT NULL
anio_nacimiento	Año de nacimiento	birthYear (de name.basics.tsv.gz)	INTEGER
anio_fallecimiento	Año de fallecimiento	deathYear (de name.basics.tsv.gz)	INTEGER

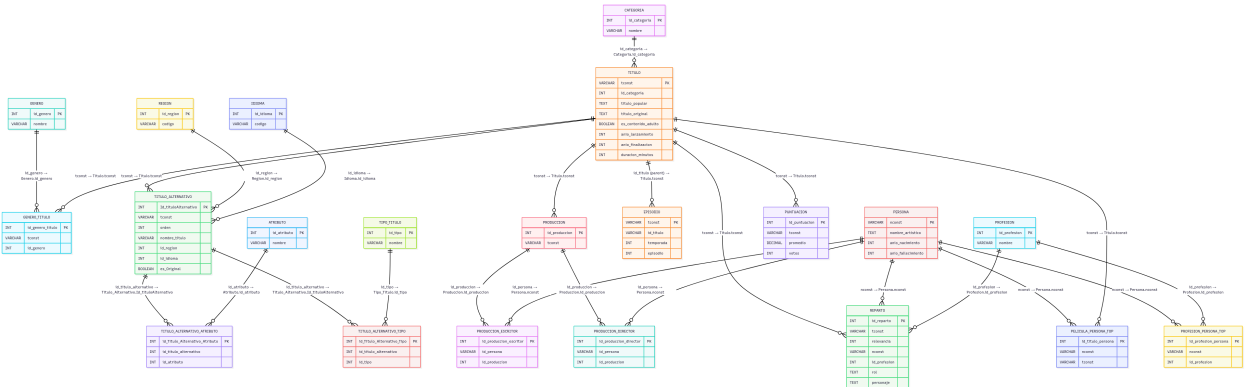
Profesion_Persona_Top

Campo	Descripción del Campo	Nombre Original en Inglés	Tipo de Datos
id_profesion_persona	Identificador único (PK)	Campo artificial - no existe en archivos	SERIAL PRIMARY KEY
nconst	Referencia a persona (FK)	nconst (de name.basics.tsv.gz)	VARCHAR(20) REFERENCES Persona(nconst)
id_profesion	Referencia a profesión (FK)	primaryProfession (procesado de name.basics.tsv.gz)	INTEGER REFERENCES Profesion(id_profesion)

Pelicula_Persona_Top

Campo	Descripción del Campo	Nombre Original en Inglés	Tipo de Datos
id_titulo_persona	Identificador único (PK)	Campo artificial - no existe en archivos	SERIAL PRIMARY KEY
nconst	Referencia a persona (FK)	nconst (de name.basics.tsv.gz)	VARCHAR(20) REFERENCES Persona(nconst)
tconst	Referencia a título (FK)	knownForTitles (procesado de name.basics.tsv.gz)	VARCHAR(20) REFERENCES Titulo(tconst)

Diagrama Entidad Relación



[illegible][illegible]

```
CREATE DATABASE imdb_database;

-- =====
-- TABLAS INDEPENDIENTES (SIN FK)
-- =====
```

```

-- Tabla Categoria
CREATE TABLE Categoria (
    id_categoria SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE
);

-- Tabla Genero
CREATE TABLE Genero (
    id_genero SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE
);

-- Tabla Region
CREATE TABLE Region (
    id_region SERIAL PRIMARY KEY,
    codigo VARCHAR(5) NOT NULL UNIQUE
);

-- Tabla Idioma
CREATE TABLE Idioma (
    id_idioma SERIAL PRIMARY KEY,
    codigo VARCHAR(5) NOT NULL UNIQUE
);

-- Tabla Tipo_Titulo
CREATE TABLE Tipo_Titulo (
    id_tipo SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE
);

-- Tabla Atributo
CREATE TABLE Atributo (
    id_atributo SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE
);

-- Tabla Profesion
CREATE TABLE Profesion (
    id_profesion SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE
);

-- Tabla Persona
CREATE TABLE Persona (
    nconst VARCHAR(20) PRIMARY KEY,
    nombre_artistico TEXT NOT NULL,
    anio_nacimiento INTEGER,
    anio_fallecimiento INTEGER,
    CONSTRAINT chk_anios CHECK (anio_fallecimiento IS NULL OR anio_fallecimiento >
= anio_nacimiento)
);

-- =====

```

```

-- TABLAS PRINCIPALES CON FK
-- =====

-- Tabla Titulo
CREATE TABLE Titulo (
    tconst VARCHAR(20) PRIMARY KEY,
    id_categoria INTEGER NOT NULL REFERENCES Categoria(id_categoria),
    titulo_popular TEXT NOT NULL,
    titulo_original TEXT,
    es_contenido_adulto BOOLEAN DEFAULT FALSE,
    anio_lanzamiento INTEGER,
    anio_finalizacion INTEGER,
    duracion_minutos INTEGER,
    CONSTRAINT chk_anios_titulo CHECK (anio_finalizacion IS NULL OR anio_finalizacion >= anio_lanzamiento),
    CONSTRAINT chk_duracion CHECK (duracion_minutos IS NULL OR duracion_minutos > 0)
);

-- Tabla Titulo_Alternativo
CREATE TABLE Titulo_Alternativo (
    id_titulo_alternativo SERIAL PRIMARY KEY,
    tconst VARCHAR(20) NOT NULL REFERENCES Titulo(tconst) ON DELETE CASCADE,
    orden INTEGER NOT NULL,
    nombre_titulo VARCHAR(255) NOT NULL,
    id_region INTEGER REFERENCES Region(id_region),
    id_idioma INTEGER REFERENCES Idioma(id_idioma),
    es_original BOOLEAN DEFAULT FALSE,
    CONSTRAINT uk_titulo_orden UNIQUE(tconst, orden)
);

-- Tabla Produccion
CREATE TABLE Produccion (
    id_produccion SERIAL PRIMARY KEY,
    tconst VARCHAR(20) NOT NULL UNIQUE REFERENCES Titulo(tconst) ON DELETE CASCADE
);

-- Tabla Episodio
CREATE TABLE Episodio (
    tconst VARCHAR(20) PRIMARY KEY REFERENCES Titulo(tconst) ON DELETE CASCADE,
    id_titulo VARCHAR(20) NOT NULL REFERENCES Titulo(tconst),
    temporada INTEGER,
    episodio INTEGER,
    CONSTRAINT chk_temporada CHECK (temporada IS NULL OR temporada > 0),
    CONSTRAINT chk_episodio CHECK (episodio IS NULL OR episodio > 0)
);

-- Tabla Reparto
CREATE TABLE Reparto (
    id_reparto SERIAL PRIMARY KEY,
    tconst VARCHAR(20) NOT NULL REFERENCES Titulo(tconst) ON DELETE CASCADE,

```

```

relevancia INTEGER NOT NULL,
nconst VARCHAR(20) NOT NULL REFERENCES Persona(nconst),
id_profesion INTEGER NOT NULL REFERENCES Profesion(id_profesion),
rol TEXT,
personaje TEXT,
CONSTRAINT uk_reparto_orden UNIQUE(tconst, relevancia),
CONSTRAINT chk_relevancia CHECK (relevancia > 0)
);

-- Tabla Puntuacion
CREATE TABLE Puntuacion (
    id_puntuacion SERIAL PRIMARY KEY,
    tconst VARCHAR(20) NOT NULL UNIQUE REFERENCES Titulo(tconst) ON DELETE CA
SCADE,
    promedio DECIMAL(3,1) NOT NULL,
    votos INTEGER NOT NULL,
    CONSTRAINT chk_promedio CHECK (promedio >= 1.0 AND promedio <= 10.0),
    CONSTRAINT chk_votos CHECK (votos > 0)
);

-- =====
-- TABLAS DE RELACIÓN (MUCHOS A MUCHOS)
-- =====

-- Tabla Genero_Titulo
CREATE TABLE Genero_Titulo (
    id_genero_titulo SERIAL PRIMARY KEY,
    tconst VARCHAR(20) NOT NULL REFERENCES Titulo(tconst) ON DELETE CASCADE,
    id_genero INTEGER NOT NULL REFERENCES Genero(id_genero),
    CONSTRAINT uk_genero_titulo UNIQUE(tconst, id_genero)
);

-- Tabla Titulo_Alternativo_Atributo
CREATE TABLE Titulo_Alternativo_Atributo (
    id_titulo_alternativo_atributo SERIAL PRIMARY KEY,
    id_titulo_alternativo INTEGER NOT NULL REFERENCES Titulo_Alternativo(id_titulo_alt
ernativo) ON DELETE CASCADE,
    id_atributo INTEGER NOT NULL REFERENCES Atributo(id_atributo),
    CONSTRAINT uk_alt_titulo_atributo UNIQUE(id_titulo_alternativo, id_atributo)
);

-- Tabla Titulo_Alternativo_Tipo
CREATE TABLE Titulo_Alternativo_Tipo (
    id_titulo_alternativo_tipo SERIAL PRIMARY KEY,
    id_titulo_alternativo INTEGER NOT NULL REFERENCES Titulo_Alternativo(id_titulo_alt
ernativo) ON DELETE CASCADE,
    id_tipo INTEGER NOT NULL REFERENCES Tipo_Titulo(id_tipo),
    CONSTRAINT uk_alt_titulo_tipo UNIQUE(id_titulo_alternativo, id_tipo)
);

-- Tabla Produccion_Escritor
CREATE TABLE Produccion_Escritor (
    id_produccion_escritor SERIAL PRIMARY KEY,

```

```

        id_persona VARCHAR(20) NOT NULL REFERENCES Persona(nconst),
        id_produccion INTEGER NOT NULL REFERENCES Produccion(id_produccion) ON DE
LETE CASCADE,
        CONSTRAINT uk_produccion_escritor UNIQUE(id_persona, id_produccion)
    );

-- Tabla Produccion_Director
CREATE TABLE Produccion_Director (
    id_produccion_director SERIAL PRIMARY KEY,
    id_persona VARCHAR(20) NOT NULL REFERENCES Persona(nconst),
    id_produccion INTEGER NOT NULL REFERENCES Produccion(id_produccion) ON DE
LETE CASCADE,
    CONSTRAINT uk_produccion_director UNIQUE(id_persona, id_produccion)
);

-- Tabla Profesion_Persona_Top
CREATE TABLE Profesion_Persona_Top (
    id_profesion_persona SERIAL PRIMARY KEY,
    nconst VARCHAR(20) NOT NULL REFERENCES Persona(nconst) ON DELETE CASCA
DE,
    id_profesion INTEGER NOT NULL REFERENCES Profesion(id_profesion),
    CONSTRAINT uk_profesion_persona UNIQUE(nconst, id_profesion)
);

-- Tabla Pelicula_Persona_Top
CREATE TABLE Pelicula_Persona_Top (
    id_titulo_persona SERIAL PRIMARY KEY,
    nconst VARCHAR(20) NOT NULL REFERENCES Persona(nconst) ON DELETE CASCA
DE,
    tconst VARCHAR(20) NOT NULL REFERENCES Titulo(tconst),
    CONSTRAINT uk_titulo_persona UNIQUE(nconst, tconst)
);

-- =====
-- ÍNDICES PARA OPTIMIZACIÓN
-- =====

-- Índices en campos de búsqueda frecuente
CREATE INDEX idx_titulo_categoria ON Titulo(id_categoria);
CREATE INDEX idx_titulo_anio ON Titulo(anio_lanzamiento);
CREATE INDEX idx_titulo_adulto ON Titulo(es_contenido_adulto);
CREATE INDEX idx_titulo_popular ON Titulo(titulo_popular);

CREATE INDEX idx_persona_nombre ON Persona(nombre_artistico);
CREATE INDEX idx_persona_nacimiento ON Persona(anio_nacimiento);

CREATE INDEX idx_reparto_titulo ON Reparto(tconst);
CREATE INDEX idx_reparto_persona ON Reparto(nconst);
CREATE INDEX idx_reparto_profesion ON Reparto(id_profesion);
CREATE INDEX idx_reparto_relevancia ON Reparto(relevancia);

CREATE INDEX idx_genero_titulo_genero ON Genero_Titulo(id_genero);
CREATE INDEX idx_genero_titulo_titulo ON Genero_Titulo(tconst);

```

```
CREATE INDEX idx_episodio_serie ON Episodio(id_titulo);
CREATE INDEX idx_episodio_temporada ON Episodio(temporada);

CREATE INDEX idx_puntuacion_promedio ON Puntuacion(promedio DESC);
CREATE INDEX idx_puntuacion_votos ON Puntuacion(votos DESC);
```

Creación Base De Datos (Utilizando Docker)

```
version: '3.8'

services:
  postgres:
    image: postgres:15-alpine
    container_name: imdb_postgres
    environment:
      POSTGRES_DB: imdb_database
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./bd.sql:/docker-entrypoint-initdb.d/01-init.sql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d imdb_database"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 30s
    restart: unless-stopped

volumes:
  postgres_data:
  pgadmin_data:
```

Separando Data

```
import pandas as pd
import os
import sys
from pathlib import Path
import logging

# Configurar logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

class IMDBDataSeparator:
```

```

def __init__(self, input_folder="../Datos", output_folder="datos_separados"):
    self.input_folder = Path(input_folder)
    self.output_folder = Path(output_folder)
    self.chunk_size = 10000

    # Crear carpeta de salida
    self.output_folder.mkdir(exist_ok=True)

def clean_value(self, value):
    """Limpia valores nulos y los convierte a None"""
    if pd.isna(value) or str(value).strip() in ['\\N', 'N/A', '', 'NULL']:
        return None
    return str(value).strip()

def split_list_field(self, value, separator=','):
    """Divide campos que contienen listas separadas por comas"""
    if not value or value is None:
        return []
    return [item.strip() for item in str(value).split(separator) if item.strip()]

def process_name_basics(self):
    """Procesa name.basics.tsv para generar datos de personas y profesiones"""
    logger.info("Procesando name.basics.tsv...")

    input_file = self.input_folder / "name.basics.tsv" / "name.basics.tsv"
    if not input_file.exists():
        logger.error(f"Archivo no encontrado: {input_file}")
        return

    # Archivos de salida
    personas_file = self.output_folder / "personas.csv"
    profesiones_file = self.output_folder / "profesiones.csv"
    profesiones_persona_file = self.output_folder / "profesiones_persona.csv"
    titulos_persona_file = self.output_folder / "titulos_persona_top.csv"

    profesiones_set = set()
    personas_data = []
    profesiones_persona_data = []
    titulos_persona_data = []

    # Procesar por chunks
    chunk_num = 0
    for chunk in pd.read_csv(input_file, sep='\t', chunksize=self.chunk_size, dtype=st
r):
        chunk_num += 1
        logger.info(f"Procesando chunk {chunk_num} de name.basics...")

        for _, row in chunk.iterrows():
            nconst = self.clean_value(row['nconst'])
            if not nconst:
                continue

            # Datos de persona

```

```

        persona_data = {
            'nconst': nconst,
            'nombre_artistico': self.clean_value(row['primaryName']),
            'anio_nacimiento': self.clean_value(row['birthYear']),
            'anio_fallecimiento': self.clean_value(row['deathYear'])
        }
        personas_data.append(persona_data)

    # Profesiones
    profesiones_raw = self.clean_value(row['primaryProfession'])
    if profesiones_raw:
        profesiones_lista = self.split_list_field(profesiones_raw)
        for profesion in profesiones_lista:
            profesiones_set.add(profesion)
            profesiones_persona_data.append({
                'nconst': nconst,
                'profesion': profesion
            })

    # Títulos conocidos
    titulos_raw = self.clean_value(row['knownForTitles'])
    if titulos_raw:
        titulos_lista = self.split_list_field(titulos_raw)
        for titulo in titulos_lista:
            titulos_persona_data.append({
                'nconst': nconst,
                'tconst': titulo
            })

    # Guardar archivos
    pd.DataFrame(personas_data).to_csv(personas_file, index=False)
    pd.DataFrame([{'nombre': prof} for prof in profesiones_set]).to_csv(profesiones_file, index=False)
    pd.DataFrame(profesiones_persona_data).to_csv(profesiones_persona_file, index=False)
    pd.DataFrame(titulos_persona_data).to_csv(titulos_persona_file, index=False)

    logger.info(f"name.basics procesado: {len(personas_data)} personas, {len(profesiones_set)} profesiones")

def process_title_basics(self):
    """Procesa title.basics.tsv para generar títulos, categorías y géneros"""
    logger.info("Procesando title.basics.tsv...")

    input_file = self.input_folder / "title.basics.tsv" / "title.basics.tsv"
    if not input_file.exists():
        logger.error(f"Archivo no encontrado: {input_file}")
        return

    # Archivos de salida
    titulos_file = self.output_folder / "titulos.csv"
    categorias_file = self.output_folder / "categorias.csv"
    generos_file = self.output_folder / "generos.csv"

```



```

generos_titulo_file = self.output_folder / "generos_titulo.csv"

categorias_set = set()
generos_set = set()
titulos_data = []
generos_titulo_data = []

chunk_num = 0
for chunk in pd.read_csv(input_file, sep='\t', chunksize=self.chunk_size, dtype=str):
    chunk_num += 1
    logger.info(f"Procesando chunk {chunk_num} de title.basics...")

    for _, row in chunk.iterrows():
        tconst = self.clean_value(row['tconst'])
        if not tconst:
            continue

        categoria = self.clean_value(row['titleType'])
        if categoria:
            categorias_set.add(categoria)

    # Datos de título
    titulo_data = {
        'tconst': tconst,
        'categoria': categoria,
        'titulo_popular': self.clean_value(row['primaryTitle']),
        'titulo_original': self.clean_value(row['originalTitle']),
        'es_contenido_adulto': self.clean_value(row['isAdult']),
        'anio_lanzamiento': self.clean_value(row['startYear']),
        'anio_finalizacion': self.clean_value(row['endYear']),
        'duracion_minutos': self.clean_value(row['runtimeMinutes'])
    }
    titulos_data.append(titulo_data)

    # Géneros
    generos_raw = self.clean_value(row['genres'])
    if generos_raw:
        generos_lista = self.split_list_field(generos_raw)
        for genero in generos_lista:
            generos_set.add(genero)
            generos_titulo_data.append({
                'tconst': tconst,
                'genero': genero
            })

    # Guardar archivos
    pd.DataFrame(titulos_data).to_csv(titulos_file, index=False)
    pd.DataFrame([{'nombre': cat} for cat in categorias_set]).to_csv(categorias_file, index=False)
    pd.DataFrame([{'nombre': gen} for gen in generos_set]).to_csv(generos_file, index=False)
    pd.DataFrame(generos_titulo_data).to_csv(generos_titulo_file, index=False)

```

```

        logger.info(f"title.basics procesado: {len(titulos_data)} títulos, {len(categorias_set)} categorías")

def process_title_akas(self):
    """Procesa title.akas.tsv para títulos alternativos"""
    logger.info("Procesando title.akas.tsv...")

    input_file = self.input_folder / "title.akas.tsv" / "title.akas.tsv"
    if not input_file.exists():
        logger.error(f"Archivo no encontrado: {input_file}")
        return

    # Archivos de salida
    titulos_alt_file = self.output_folder / "titulos_alternativos.csv"
    regiones_file = self.output_folder / "regiones.csv"
    idiomas_file = self.output_folder / "idiomas.csv"
    tipos_titulo_file = self.output_folder / "tipos_titulo.csv"
    atributos_file = self.output_folder / "atributos.csv"

    regiones_set = set()
    idiomas_set = set()
    tipos_set = set()
    atributos_set = set()
    titulos_alt_data = []

    chunk_num = 0
    for chunk in pd.read_csv(input_file, sep='\t', chunksize=self.chunk_size, dtype=str):
        chunk_num += 1
        logger.info(f"Procesando chunk {chunk_num} de title.akas...")

        for _, row in chunk.iterrows():
            tconst = self.clean_value(row['titleId'])
            if not tconst:
                continue

            region = self.clean_value(row['region'])
            idioma = self.clean_value(row['language'])
            tipos_raw = self.clean_value(row['types'])
            atributos_raw = self.clean_value(row['attributes'])

            if region:
                regiones_set.add(region)
            if idioma:
                idiomas_set.add(idioma)

    # Tipos
    tipos_lista = []
    if tipos_raw:
        tipos_lista = self.split_list_field(tipos_raw)
        for tipo in tipos_lista:
            tipos_set.add(tipo)

```

```

# Atributos
atributos_lista = []
if atributos_raw:
    atributos_lista = self.split_list_field(atributos_raw)
    for atributo in atributos_lista:
        atributos_set.add(atributo)

titulo_alt_data = {
    'tconst': tconst,
    'orden': self.clean_value(row['ordering']),
    'nombre_titulo': self.clean_value(row['title']),
    'region': region,
    'idioma': idioma,
    'tipos': ','.join(tipos_lista) if tipos_lista else None,
    'atributos': ','.join(atributos_lista) if atributos_lista else None,
    'es_original': self.clean_value(row['isOriginalTitle'])
}
titulos_alt_data.append(titulo_alt_data)

# Guardar archivos
pd.DataFrame(titulos_alt_data).to_csv(titulos_alt_file, index=False)
pd.DataFrame([{'codigo': reg} for reg in regiones_set]).to_csv(regiones_file, index=False)
pd.DataFrame([{'codigo': idi} for idi in idiomas_set]).to_csv(idiomas_file, index=False)
pd.DataFrame([{'nombre': tipo} for tipo in tipos_set]).to_csv(tipos_titulo_file, index=False)
pd.DataFrame([{'nombre': attr} for attr in atributos_set]).to_csv(atributos_file, index=False)

logger.info(f"title.akas procesado: {len(titulos_alt_data)} títulos alternativos")

def process_title_crew(self):
    """Procesa title.crew.tsv para directores y escritores"""
    logger.info("Procesando title.crew.tsv...")

    input_file = self.input_folder / "title.crew.tsv" / "title.crew.tsv"
    if not input_file.exists():
        logger.error(f"Archivo no encontrado: {input_file}")
        return

    directores_data = []
    escritores_data = []
    producciones_data = []

    chunk_num = 0
    for chunk in pd.read_csv(input_file, sep='\t', chunksize=self.chunk_size, dtype=str):
        chunk_num += 1
        logger.info(f"Procesando chunk {chunk_num} de title.crew...")

        for _, row in chunk.iterrows():

```

```

        tconst = self.clean_value(row['tconst'])
        if not tconst:
            continue

        producciones_data.append({'tconst': tconst})

    # Directores
    directores_raw = self.clean_value(row['directors'])
    if directores_raw:
        directores_lista = self.split_list_field(directores_raw)
        for director in directores_lista:
            directores_data.append({
                'tconst': tconst,
                'nconst': director
            })

    # Escritores
    escritores_raw = self.clean_value(row['writers'])
    if escritores_raw:
        escritores_lista = self.split_list_field(escritores_raw)
        for escritor in escritores_lista:
            escritores_data.append({
                'tconst': tconst,
                'nconst': escritor
            })

    # Guardar archivos
    pd.DataFrame(producciones_data).to_csv(self.output_folder / "producciones.csv",
index=False)
    pd.DataFrame(directores_data).to_csv(self.output_folder / "directores.csv", index=
False)
    pd.DataFrame(escritores_data).to_csv(self.output_folder / "escritores.csv", index=
False)

    logger.info(f"title.crew procesado: {len(directores_data)} directores, {len(escritore
s_data)} escritores")

def process_title_episode(self):
    """Procesa title.episode.tsv para episodios"""
    logger.info("Procesando title.episode.tsv...")

    input_file = self.input_folder / "title.episode.tsv" / "title.episode.tsv"
    if not input_file.exists():
        logger.error(f"Archivo no encontrado: {input_file}")
        return

    episodios_data = []

    chunk_num = 0
    for chunk in pd.read_csv(input_file, sep='\t', chunksize=self.chunk_size, dtype=st
r):
        chunk_num += 1
        logger.info(f"Procesando chunk {chunk_num} de title.episode...")

```

```

for _, row in chunk.iterrows():
    tconst = self.clean_value(row['tconst'])
    if not tconst:
        continue

    episodio_data = {
        'tconst': tconst,
        'id_titulo': self.clean_value(row['parentTconst']),
        'temporada': self.clean_value(row['seasonNumber']),
        'episodio': self.clean_value(row['episodeNumber'])
    }
    episodios_data.append(episodio_data)

pd.DataFrame(episodios_data).to_csv(self.output_folder / "episodios.csv", index=False)
logger.info(f"title.episode procesado: {len(episodios_data)} episodios")

def process_title_principals(self):
    """Procesa title.principals.tsv para reparto principal"""
    logger.info("Procesando title.principals.tsv...")

    input_file = self.input_folder / "title.principals.tsv" / "title.principals.tsv"
    if not input_file.exists():
        logger.error(f"Archivo no encontrado: {input_file}")
        return

    reparto_data = []

    chunk_num = 0
    for chunk in pd.read_csv(input_file, sep='\t', chunksize=self.chunk_size, dtype=str):
        chunk_num += 1
        logger.info(f"Procesando chunk {chunk_num} de title.principals...")

        for _, row in chunk.iterrows():
            tconst = self.clean_value(row['tconst'])
            if not tconst:
                continue

            reparto_item = {
                'tconst': tconst,
                'relevancia': self.clean_value(row['ordering']),
                'nconst': self.clean_value(row['nconst']),
                'categoria_profesion': self.clean_value(row['category']),
                'rol': self.clean_value(row['job']),
                'personaje': self.clean_value(row['characters'])
            }
            reparto_data.append(reparto_item)

    pd.DataFrame(reparto_data).to_csv(self.output_folder / "reparto.csv", index=False)
    logger.info(f"title.principals procesado: {len(reparto_data)} registros de reparto")

```

```

def process_title_ratings(self):
    """Procesa title.ratings.tsv para puntuaciones"""
    logger.info("Procesando title.ratings.tsv...")

    input_file = self.input_folder / "title.ratings.tsv" / "title.ratings.tsv"
    if not input_file.exists():
        logger.error(f"Archivo no encontrado: {input_file}")
        return

    puntuaciones_data = []

    chunk_num = 0
    for chunk in pd.read_csv(input_file, sep='\t', chunksize=self.chunk_size, dtype=st
r):
        chunk_num += 1
        logger.info(f"Procesando chunk {chunk_num} de title.ratings...")

        for _, row in chunk.iterrows():
            tconst = self.clean_value(row['tconst'])
            if not tconst:
                continue

            puntuacion_data = {
                'tconst': tconst,
                'promedio': self.clean_value(row['averageRating']),
                'votos': self.clean_value(row['numVotes'])
            }
            puntuaciones_data.append(puntuacion_data)

    pd.DataFrame(puntuaciones_data).to_csv(self.output_folder / "puntuaciones.csv", i
ndex=False)
    logger.info(f"title.ratings procesado: {len(puntuaciones_data)} puntuaciones")

def process_all_files(self):
    """Procesa todos los archivos"""
    logger.info("Iniciando procesamiento de todos los archivos IMDB...")

    try:
        self.process_name_basics()
        self.process_title_basics()
        self.process_title_akas()
        self.process_title_crew()
        self.process_title_episode()
        self.process_title_principals()
        self.process_title_ratings()

        logger.info("¡Procesamiento completado exitosamente!")
        logger.info(f"Archivos generados en la carpeta: {self.output_folder}")

    except Exception as e:
        logger.error(f"Error durante el procesamiento: {str(e)}")
        raise

```

```
if __name__ == "__main__":  
    # Crear instancia del separador  
    separator = IMDBDataSeparator()  
  
    # Procesar todos los archivos  
    separator.process_all_files()
```